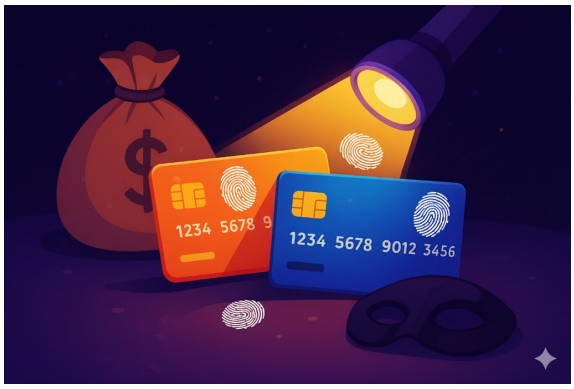
Metodología de la Programación

Curso 2025/2026



Guion de prácticas

Fraud3

Clases con memoria dinámica. Reducción de la dimensionalidad de un dataset

Abril de 2026

Silvia Acid, Andrés Cano, Luis Castillo
Dpto. Ciencias de la Computación e Inteligencia Artificial
Universidad de Granada

Licencia Creative Commons con Reconocimiento - No Comercial - Compartir Igual 4.0 (CC BY-NC-SA 4.0)

Índice

1. Introducción **5**

2. Arquitectura de las prácticas **5**

3. Objetivos **6**

4. Datasets. Reducción de la dimensionalidad **7**
4.1. Datasets 7
4.2. Ficheros dataset 8
4.3. Reducción de la dimensionalidad de un dataset 9
4.4. Algoritmo de reducción de la dimensionalidad 10

5. Práctica a entregar **11**
5.1. Finalidad del programa 11
5.2. Sintaxis y ejemplos de ejecución 11
5.3. Módulos del proyecto 15
5.4. Entrega de la práctica 19

6. Código para la práctica **20**
6.1. Location.h 20
6.2. VectorLocation.h 23
6.3. VectorInt.h 26
6.4. Clustering.h 28
6.5. DataSet.h 32
6.6. main.cpp 36

1. Introducción

En esta práctica descartamos el módulo `ArrayClustering`, que ya no necesitamos. Conservamos el resto de las clases, aunque las clases `VectorLocation` y `VectorInt` deben ser refactorizadas para que los arrays que contienen se alojen en memoria dinámica, en lugar de en memoria automática (la pila). Este cambio provocará que tengamos que revisar gran parte de sus métodos. No será necesario revisar el resto de clases.

Añadiremos además la clase `DataSet`, que nos servirá para guardar un conjunto de instancias, cada una de las cuales servirá para almacenar el importe de cada una de las compras que realiza un cliente en un conjunto de localizaciones. Así, un dataset puede verse como una matriz bidimensional con tantas filas como instancias haya en el dataset y tantas columnas como número de localizaciones tenga. La matriz bidimensional será almacenada también en memoria dinámica.

La finalidad del programa de esta práctica es resumir el conjunto de localizaciones de un dataset para que use un conjunto más pequeño. O sea, el programa reducirá el número de columnas del dataset como se detallará más adelante.

2. Arquitectura de las prácticas

Como indicamos en la Introducción, en `Fraud3` desaparece el módulo `ArrayClustering`, y se refactorizan las clases `VectorLocation` y `VectorInt`. Las clases `Location` y `Clustering` no necesitan en principio ningún cambio. Como novedad, tenemos la clase `DataSet`, que nos permite almacenar un dataset. Como siempre, puede ver en la figura 1 la lista de módulos que usaremos.

Describamos brevemente los módulos que usaremos en esta práctica:

A Location
La clase Location se implementó en prácticas anteriores y no necesita de ningún cambio en esta práctica.

B VectorLocation
Necesita una importante refactorización para usar un vector dinámico de objetos `Location` en lugar de uno automático (ver sección 5.3).

C VectorInt
También necesita una importante refactorización para usar un vector dinámico de enteros, en lugar de uno automático (ver sección 5.3).

D Clustering
La clase Clustering no necesita de ningún cambio en esta práctica.

E DataSet
Nueva clase que se utiliza para almacenar un dataset. Usa una ma-

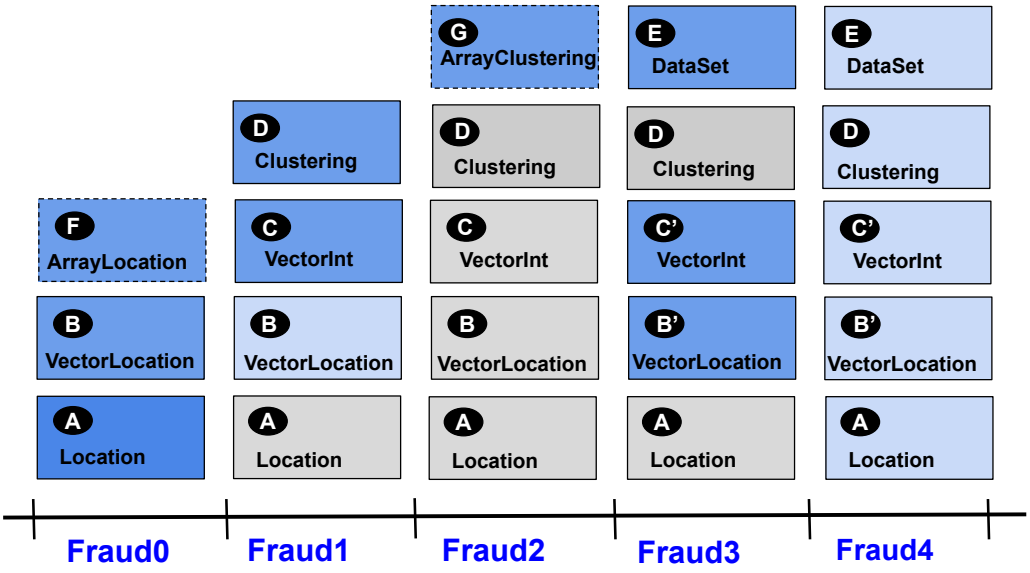


Figura 1: Arquitectura de las prácticas de MP 2026. Como podemos observar, los bloques correspondientes a cada módulo, aparecen en distintos colores. El color azul indica software que requiere ser desarrollado por el estudiante. Los cambios considerables (como los cambios en la estructura interna de una clase) se muestran en color intenso, mientras que pequeños cambios, como la incorporación de nuevas funcionalidades, se muestran en su correspondiente color suave. Finalmente, en gris se muestran módulos que no sufren cambios respecto a la versión anterior de las prácticas.

triz dinámica bidimensional para almacenar el conjunto de instancias del dataset.

main.cpp Contiene la función main() cuya finalidad será la de reducir la dimensionalidad de un dataset leído de un fichero `dts`, guardando el resultado en otro fichero `dts` de salida.

3. Objetivos

El desarrollo de la práctica `Fraud3` persigue los siguientes objetivos:

- Aprender a desarrollar los métodos básicos de una clase que contenga memoria dinámica: constructor de copia, destructor y operador de asignación.
- Refactorizar las clases `VectorInt` y `VectorLocation` para que usen vectores en memoria dinámica, en lugar de vectores en la pila, tratando de reducir al mínimo el número de cambios a realizar en los métodos que tenían tales clases.
- Construir la clase `DataSet` que contiene una matriz bidimensional en memoria dinámica.

- Seguir practicando con la lectura de los parámetros pasados a un programa por la línea de comandos.
- Profundizar en la lectura/escritura de datos de ficheros de texto.
- Ahondar en el uso del depurador de VSCode y `valgrind` para rastrear errores en tiempo de ejecución.

4. Datasets. Reducción de la dimensionalidad

En esta sección explicaremos qué es un dataset, el formato de fichero que usaremos para almacenar un dataset y el algoritmo para reducir la dimensión de un dataset.

4.1. Datasets

En esta práctica vamos a trabajar con datos sobre el importe de las compras que realizan un conjunto de clientes en un determinado conjunto de localizaciones. A este tipo de datos le llamaremos *dataset*. Un dataset contiene la siguiente información:

- Información sobre un conjunto de m localizaciones. Cada localización contiene las coordenadas `x` e `y`, y nombre de la localización.
- Conjunto de n instancias. Cada instancia se corresponde con las compras realizadas por un determinado cliente. La instancia almacena el importe de cada una de las compras que ha realizado tal cliente en cada una de las m localizaciones. En esta práctica usaremos números enteros para guardar tales importes.
- Etiqueta de cada una de las instancias. La etiqueta indica la clase asociada a la instancia correspondiente. En la práctica usaremos también números enteros como valores de las etiquetas.

El conjunto de instancias del dataset con los datos sobre las compras de los clientes, podemos verlo como una matriz bidimensional con tantas filas como instancias tenga el dataset y tantas columnas como número de localizaciones. El ejemplo 1 muestra la matriz de un dataset, junto con la etiqueta de cada instancia.

Ejemplo 1 *Ilustración parcial del contenido de un dataset*

La siguiente tabla muestra un dataset con 5 instancias y 6 localizaciones. La columna de más a la derecha muestra la etiqueta de cada instancia. No mostramos aquí los datos sobre las localizaciones de este dataset.

	0	1	2	3	4	5	Etiq.
0	0	100	0	200	1	2	1
1	15	3	1	2	30	1	0
2	2	250	0	100	2	4	1
3	0	3	0	2	15	2	0
4	5	3	1	2	40	1	0

Usaremos objetos de la clase `DataSet` para guardar los datos de un dataset. En la sección 5.3 se detallan los datos miembro que tiene esta clase para almacenar el conjunto de localizaciones asociado al dataset, las instancias, y la etiqueta de cada una de las instancias.

4.2. Ficheros dataset

Usaremos un fichero con extensión `.dts` para guardar un dataset. Un fichero `dts` es un fichero de texto que contiene información acerca del conjunto de localizaciones del dataset y la información del importe de las compras de cada cliente en el conjunto de localizaciones. Un fichero `dts` tiene el siguiente formato:

- La primera línea contiene `MP-FRAUD_DATASET-T-1.0`, una cadena mágica que indica que el fichero contiene un dataset en formato texto, para las prácticas Fraud de la asignatura MP (Metodología de la Programación).
- Un número entero positivo con el número m de localizaciones que vienen a continuación.
- Las siguientes líneas contienen la información de cada una de las m localizaciones. Cada línea contiene la información de una localización (coordenada x, coordenada y, nombre). Los campos de cada localización están separados con uno o varios caracteres blanco. El campo nombre podría estar compuesto de varias palabras separadas por un espacio en blanco. También es posible que el campo nombre esté vacío. Es posible que aparezcan uno o varios espacios en blanco antes o después del valor de cada campo, de la misma forma que ocurría en Fraud2.
- Un número entero con el número de instancias del dataset.
- Un conjunto de n números enteros con la etiqueta de cada una de las instancias. Se usa uno o varios espacios en blanco como separador entre etiquetas.
- Finalmente, aparecen n líneas (una para cada instancia), cada una de las cuales contiene m enteros que representan el importe gastado por un determinado cliente en cada una de las m localizaciones.

Ejemplo 2 *Ejemplo de fichero dataset con 5 instancias y 6 localizaciones: DataSets/princeton_6locs.dts*

El siguiente contenido corresponde al contenido de un fichero `dts` *con un dataset con 5 instancias y 6 localizaciones. Corresponde con el dataset del ejemplo 1.*

```
MP-FRAUD_DATASET-T-1.0
6
11.8 6.9 Wawa Public Restrooms
24.8 14.9 Quadrangle
12.6 7.3 Princeton Station
25.6 14.9 Ivy
```

```
10.1 9.7 Forbes College
13.3 12.2 Dinky Bar & Kitchen
5
1 0 1 0 0
  0 100   0 200   1   2
 15   3   1   2  30   1
  2 250   0 100   2   4
  0   3   0   2  15   2
  5   3   1   2  40   1
```

4.3. Reducción de la dimensionalidad de un dataset

La finalidad principal del programa que haremos en esta práctica es la reducción de la dimensionalidad de un dataset. Reducir la dimensionalidad de un dataset significa reducir el número de variables del dataset. O sea, significa reducir el número de columnas, manteniendo igual el número de filas y las etiquetas de cada instancia.

En esta práctica aplicaremos un proceso de reducción de la dimensionalidad de un dataset guiado por un clustering obtenido a partir del conjunto de localizaciones del dataset y del parámetro K. El dataset resultado tendrá K columnas (número de clústeres usados en el clustering). Cada columna del dataset resultado se obtendrá como suma de las columnas correspondientes a localizaciones incluídas en el mismo clúster. De esta forma, la columna 0 del dataset resultado tendrá la suma de las columnas correspondientes a las localizaciones del clúster 0, la columna 1 la suma de columnas de las columnas correspondientes a las localizaciones del clúster 1, y así sucesivamente.

El conjunto de localizaciones del dataset reducido se obtendrá como copia del conjunto de centroides del clustering empleado. Del mismo modo, el conjunto de etiquetas del dataset resultado será una copia del conjunto de etiquetas del dataset original.

Ejemplo 3 *Ejemplo de reducción de la dimensionalidad de un dataset*

Supongamos el dataset del ejemplo 1 que replicamos de nuevo a continuación:

	0	1	2	3	4	5	Etiq.
0	0	100	0	200	1	2	1
1	15	3	1	2	30	1	0
2	2	250	0	100	2	4	1
3	0	3	0	2	15	2	0
4	5	3	1	2	40	1	0

Supongamos que hemos realizado un proceso de clustering usando $K = 2$ para el conjunto de localizaciones del dataset y hemos obtenido los siguientes clústeres:

- *Clúster 0:* $\{0, 2, 4, 5\}$
- *Clúster 1:* $\{1, 3\}$

Por tanto, el dataset resultado tendrá 5 filas y 2 columnas. La columna 0 se obtendrá como suma de las columnas $\{0, 2, 4, 5\}$ y la columna 1 como suma de las columnas $\{1, 3\}$. Este dataset sería el siguiente:

	0	1	Etiq.
0	3	300	1
1	47	5	0
2	8	350	1
3	17	5	0
4	47	5	0

Como vemos, las etiquetas de cada instancia del dataset resultado son las mismas que las del dataset original.

4.4. Algoritmo de reducción de la dimensionalidad

El algoritmo que obtiene un dataset de dimensionalidad reducida lo implementaremos en el método `getReducedDataSet()` de la clase `Dataset`. A este método se le pasa un objeto `Clustering` y devolverá un nuevo dataset de dimensionalidad reducida obtenido a partir del dataset del objeto implícito y el objeto `Clustering`. Como puede ver en el fichero `DataSet.h`, este método tiene la especificación que se muestra más abajo, aunque debe revisar la forma correcta de pasar su parámetro, y si debe o no ser un método constante.

```
class DataSet {
public:
...
   /**
     * @brief Gets a new DataSet from this DataSet. The DataSet will contain
     * the same number of instances as this DataSet, but the number of locations
     * is given by the number of clusters in the provided Clustering object.
     * Therefore, the returned DataSet will have a reduced number of columns
     * and an equal number of rows compared to this DataSet.
     * The provided Clustering object defines how to reduce the set of columns.
     * Column 0 will contain the sum of the columns of this DataSet belonging
     * to cluster 0, column 1 the sum of the columns belonging to cluster 1,
     * and so on.
     * The vector of locations will be a copy of the vector of centroids of the
     * provided Clustering object.
     * The vector of labels will be a copy of the vector of labels in this
     * DataSet.
     * @throw throw std::invalid_argument Throws a std::invalid_argument
     * exception if the method Clustering::isDone() return false with the
     * provided Clustering object.
     * @throw throw std::invalid_argument Throws a std::invalid_argument
     * exception if the number of locations in the provided Clustering object
     * is not equal to the number of locations in this DataSet.
     * Query method
     * @param clustering A Clustering object.
     * Input parameter
     * @return A reduced DataSet
     */
    DataSet getReducedDataSet(Clustering clustering);
...
}  // end of class DataSet
```

Sea ds un dataset con n instancias y m localizaciones sobre el que hemos realizado un proceso de clustering de sus localizaciones usando K clústeres. El método `getReducedDataSet()` realizará los siguientes pasos para obtener el dataset reducido ds_R:

1. Crear un nuevo dataset ds_R con n instancias (filas) y con K localizaciones (columnas), con un valor 0 en todas las casillas de la matriz bidimensional.

2. Las localizaciones de ds_R serán una copia de los centroides del clustering utilizado.

3. Las etiquetas de ds_R serán una copia de las etiquetas de ds.

4. Para cada columna col de ds

   a) Sea $cluster_num$ el número de clúster asignado a la localización número col de ds

   b) Para cada fila fil del dataset ds
   $ds_R[fil][cluster_num]$ += $ds[fil][col]$

5. Práctica a entregar

5.1. Finalidad del programa

El propósito del programa Fraud3 es leer un dataset desde un fichero `dts` de entrada y obtener a partir de él, un dataset de salida con dimensionalidad reducida, o sea con un menor número de variables (columnas en la matriz bidimensional). El dataset resultado se guardará entonces en otro fichero `dts`.

El número de clústeres K a usar en el proceso de clustering se proporcionará al programa mediante un parámetro opcional de la línea de comandos. Si este parámetro no es proporcionado, se usará el valor $K = 5$.

El nombre del fichero `dts` de salida también se proporcionará al programa con un parámetro opcional de la línea de comandos. Si no es proporcionado, se usará el nombre `tests/output/output.dts`.

El nombre del fichero `dts` de entrada se pasará al programa como último parámetro de la línea de comandos.

El programa llevará a cabo los siguientes pasos:

1. Obtener el valor de K y los nombres de los ficheros `dts` de salida y entrada a partir de los argumentos pasados a la función `main()`.

2. Leer el fichero de entrada, almacenándolo en un objeto `DataSet`.

3. Realizar un proceso de clustering, con el valor K seleccionado, para el conjunto de localizaciones del dataset de entrada. Como semilla, se usará el valor por defecto que establece el método `set()` de la clase `Clustering`.

4. Obtener un objeto `DataSet` de salida con dimensionalidad reducida, a partir del dataset de entrada y el clustering calculado en el paso anterior.

5. Salvar el objeto `DataSet` de salida en el fichero `dts` de salida.

5.2. Sintaxis y ejemplos de ejecución

Como se ha adelantado en la sección 5.1, el programa de esta práctica tiene un parámetro obligatorio, el nombre del fichero `dts` de entrada, y dos parámetros opcionales, el valor de K y el nombre del fichero `dts` de salida.

La **sintaxis de ejecución** del programa desde un terminal es:

```
Fraud3> build/Fraud3 [-K <K>] [-o <outputFile.dts>] <inputFile.dts>
```

Veamos algunos ejemplos de ejecución del programa desde la línea de comandos:

Ejemplo 4 *Falta proporcionar el nombre del fichero dts de entrada:*

```
Fraud3> build/Fraud3
```

La salida del programa será la que muestra la función `showHelp()`*, incluida en* `main.cpp`*, con el mensaje* `"Input file not provided"`*:*

```
ERROR in Fraud3 parameters. Input file not provided
Run with the following arguments:
Fraud3 [-K <K>] [-o <outputFile.dts>] <inputFile.dts>

Parameters:
-K <K>: an integer with the number of clusters to use (5 by default)
-o <outputFile.dts>: name of the output dataset file (tests/output/output.dts
↪  by default
<inputFile.dts>: name of the input dataset file
```

Ejemplo 5 *De nuevo, falta proporcionar el nombre del fichero dts de entrada:*

```
Fraud3> build/Fraud3 -K 3 -o tests/output/output.dts
```

La salida del programa será de nuevo:

```
ERROR in Fraud3 parameters. Input file not provided
Run with the following arguments:
Fraud3 [-K <K>] [-o <outputFile.dts>] <inputFile.dts>

Parameters:
-K <K>: an integer with the number of clusters to use (5 by default)
-o <outputFile.dts>: name of the output dataset file (tests/output/output.dts
↪  by default
<inputFile.dts>: name of the input dataset file
```

Ejemplo 6 *Reducción de dimensionalidad de* `princeton_6locs.dts`

Reducción de la dimensionalidad del dataset `princeton_6locs.dts`*, usando* $K = 2$ *y guardando el resultado en el fichero* `/tmp/princeton_6locs_reduced.dts`*:*

```
Fraud3> build/Fraud3 -K 2 -o /tmp/princeton_6locs_reduced.dts
↪  ../Datasets/princeton_6locs.dts
```

En este caso, no se obtiene ninguna salida por consola. El programa habrá creado el fichero `/tmp/princeton_6locs_reduced.dts` *cuyo contenido debería ser el mismo que el del fichero* `Datasets/dataP3/princeton_6locs_reduced.dts`*. Mostramos a continuación el contenido de este fichero de salida:*

```
MP-FRAUD_DATASET-T-1.0
2
11.950000 9.025000
25.200000 14.900000
5
1 0 1 0 0
3 300
47 5
8 350
17 5
47 5
```

Como podemos ver, la reducción de dimensionalidad obtenida es la que se obtuvo en el ejemplo 3.

Ejemplo 7 *Reducción de dimensionalidad de* `princeton_training.dts`

Reducción de la dimensionalidad del dataset `princeton_training.dts`*, usando* $K = 5$ *y guardando el resultado en el fichero* `/tmp/princeton_training_reduced.dts`*:*

```
Fraud3> build/Fraud3 -K 5 -o
↪  /tmp/princeton_training_reduced.dts
↪  ../Datasets/princeton_training.dts
```

De nuevo, no se obtiene ninguna salida por consola. El programa habrá creado el fichero `/tmp/princeton_training_reduced.dts`*. El contenido de este fichero debería ser el mismo que el que aparece en el fichero* `Fraud3/tests/ fraud3_test07.test`*, tras la línea* `%%%OUTPUT`*.*

Ejemplo 8 *Reducción de dimensionalidad de* `princeton_training.dts`

Reducción de la dimensionalidad del dataset `princeton_training.dts`*, sin proporcionar por la línea de comandos el valor de* K *y el nombre del fichero de salida. Ambos parámetros tomarán un valor por defecto (*$K = 5$ *y* `tests/output/output.dts` *para el fichero de salida) según se indica en la especificación de la función* `main()`*, en el fichero* `main.cpp`*:*

```
Fraud3> build/Fraud3 ../Datasets/princeton_training.dts
```

Para que la anterior ejecución de `Fraud3` *pueda crear el fichero de salida, es necesario que la carpeta* `tests/output` *esté creada previamente.*
De nuevo, no se obtiene ninguna salida por consola. El programa habrá creado el fichero `tests/output/output.dts`*. El contenido de este fichero debería ser el mismo que el que aparece en el fichero* `Fraud3/tests/fraud3_test07.test`*, tras la línea* `%%%OUTPUT`*.*

En las carpetas `CodeProjects/Datasets` y `CodeProjects/Datasets/dataP3`, puede encontrar ficheros dataset `dts` con los que podrá hacer diferentes pruebas.

Además, se proporcionan una serie de ficheros con extensión `.test` que contienen tests de integración, y que pueden usarse con el script `runTest.sh` [1]. Veamos un ejemplo de ejecución de este script:

Ejemplo 9 *Ejecución de* `runTest.sh` *con un fichero de tests:*

```
Fraud3> scripts/runTest.sh tests/fraud3_test03.test
```

La salida que debería obtenerse al ejecutar el anterior script sería la siguiente:

```
Loading ansiterminal...

VALIDATION TOOL v2.0
Checking tests folder           OK
Setting everything up           OK
Test #1  K=2, group the expenses in 2 clusters

Testing tests/fraud3_test03.test                Generating fresh binaries  -- Configuring
↪  done (0.0s)
-- Generating done (0.0s)
-- Build files have been written to:
↪  /home/acu/Latex/Docencia/MP/MP_2025_2026/Github/MP2526/CodeProjects/Fraud3Teacher/build

[  OK  ] Test #1 [tests/fraud3_test03.test] (valgrind --leak-check=full
↪  build/Fraud3Teacher  -K 2 -o  tests/output/simple_2NewLocations.dts
↪  ../Datasets/dataP3/simple_5Locations.dts )
        ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
End of tests
```

El contenido del fichero de test utilizado (`fraud3_test03.test`*) es el siguiente:*

```
%%%CALL -K 2 -o  tests/output/simple_2NewLocations.dts
↪  ../Datasets/dataP3/simple_5Locations.dts
%%%VALGRIND
%%%DESCRIPTION K=2, group the expenses in 2 clusters
%%%FROMFILE ../Datasets/dataP3/simple_2NewLocations.dts
%%%OUTPUT
MP-FRAUD_DATASET-T-1.0
2
3.500000 3.500000
1.000000 1.000000
10
0 1 1 1 0 0 0 0 0 0
9 6
2770 1476
1318 1338
1283 2850
20 40
11 11
13 154
32 113
0 30
2 3
```

La salida que aparece tras la línea `%%%OUTPUT` *es la que debería aparecer como contenido del fichero dataset de salida* `tests/output/simple_2NewLocations.dts`.

[1] Los detalles de como ejecutar tests de integración de forma automática se encuentran en el guion Fraud0.

5.3. Módulos del proyecto

A la hora de implementar los nuevos métodos y funciones de esta práctica, fíjese en las cabeceras de tales métodos y funciones en los ficheros `*.h` correspondientes y en los comentarios que les acompañan, pues en ellos están detalladas sus especificaciones.

Nota importante: Se han retirado a propósito todos los `const` y `&` para los parámetros de los nuevos métodos que aparecen en las clases `VectorInt`, `VectorLocation` y `DataSet`. También se ha retirado el calificador `const` que debe aplicarse a algunos métodos (los métodos que no modifican el objeto implícito). El número de argumentos y los tipos han sido establecidos y **no se han de cambiar**.

Se le invita a definir todos los métodos y funciones externas adicionales que estime oportuno para una adecuada modularización del código.

En esta práctica debe tener en cuenta las especificaciones indicadas en los siguientes módulos:

- Clase `Location`. En esta clase no se necesita ningún cambio respecto a la práctica anterior.
- Clase `Clustering`. En esta clase no se necesita ningún cambio respecto a la práctica anterior.
- Clase `VectorLocation`. El fichero **VectorLocation.h** (ver detalles en sección 6.2) contiene la declaración de la clase `VectorLocation`. En esta práctica hay un cambio importante en esta clase respecto a la práctica anterior. Ahora el array de objetos `Location` está alojado en memoria dinámica en lugar de en memoria automática, array que podrá aumentar de capacidad cuando el número de elementos útiles coincida con su capacidad (o sea, cuando la capacidad actual se agote). El mecanismo que usaremos para aumentar la capacidad del array, será el mismo al que se usó en `Fraud2` para aumentar la capacidad del array de objetos `Clustering` (vea la sección 5 del guion de `Fraud2`), construyendo un nuevo array con un tamaño igual al que tenía anteriormente más un espacio adicional dado por la constante `BLOCK_SIZE` (definida en `VectorLocation.h`). Describamos brevemente los datos miembro de la nueva versión de la clase:
    - `_locations`: es ahora un puntero a un array de objetos `Location`.
    - `_size`: igual que en prácticas anteriores, permite conocer en todo momento el número de elementos usados en el array.
    - `_capacity`: guarda la capacidad actual del array dinámico.
    - `BLOCK_SIZE`: constante que proporciona el número de elementos (tamaño de bloque) en que crecerá la capacidad del array actual, cuando esta se agote.

```
class VectorLocation {
...
private:
    /**
     * Pointer to a dynamic array of Locations
     */
    Location* _locations;

    /**
     * Number of Location objects contained in the dynamic array _locations
     */
    int _size;

    /**
     * Maximun number of Location objects that can be stored in the dynamic array
     * _locations
     */
    int _capacity;

    /**
     * Size of new memory blocks when resizing the dynamic array _locations
     */
    static const int BLOCK_SIZE=20;
}; // end class VectorLocation
```

En esta clase será necesario implementar o revisar los siguientes métodos o constructores:

- Constructor `VectorLocation(int size=0)`. Se debe refactorizar debido a la necesidad de reservar memoria para el array dinámico.
- Constructor de copia. Constructor que debe implementarse en aquellas clases que reservan memoria dinámica.
- Destructor. También debe implementarse en aquellas clases que reservan memoria dinámica.
- Operador de asignación. Tercer método a implementar en clases que reservan memoria dinámica.
- Método `int getCapacity()`. Ahora, la capacidad del array está almacenada en el dato miembro `_capacity`.
- Método `bool append()`. Este método debe modificarse para que realoje el array dinámico cuando se va a insertar una nueva localización y su capacidad se ha agotado. Ahora el método no lanzará una excepción cuando la capacidad del array se ha agotado, pues esto lo resuelve el método con el realojo.
- Método `void load()`. Revíselo, pues seguramente debe modificarse debido al uso de memoria dinámica en esta clase. Además, ahora solo lanza excepción cuando el número de localizaciones leídas del fichero es negativo.

Revise el resto de métodos, por si hubiese que modificar alguno más.

Además, debería implementar métodos privados que realicen las tareas de reservar la memoria dinámica, liberarla, copia y realojo, pues pueden usarse en varios de los métodos públicos de la clase. Implementando estos métodos privados, evitamos repetición de código.

- Clase `VectorInt`. El fichero **VectorInt.h** (ver detalles en sección 6.3) contiene la declaración de la clase `VectorInt`. De nuevo, esta clase presenta un cambio importante respecto a la práctica anterior.

Ahora el array de enteros está alojado en memoria dinámica en lugar de en memoria automática, array que podrá aumentar de capacidad cuando el número de elementos útiles coincida con su capacidad (o sea, cuando la capacidad actual se agote). Al igual que en `VectorLocation`, el mecanismo que usaremos para aumentar la capacidad del array, será el mismo al que se usó en `Fraud2` para aumentar la capacidad del array de objetos `Clustering` (vea la sección 5 del guion de `Fraud2`), construyendo un nuevo array con un tamaño igual al que tenía anteriormente más un espacio adicional dado por la constante `BLOCK_SIZE` (definida en `VectorInt.h`). Describamos brevemente los datos miembro de la nueva versión de la clase:

- `_values`: es ahora un puntero a un array de enteros.
- `_size`: igual que en prácticas anteriores, permite conocer en todo momento el número de elementos usados en el array.
- `_capacity`: guarda la capacidad actual del array dinámico.
- `BLOCK_SIZE`: constante que proporciona el número de elementos (tamaño de bloque) en que crecerá la capacidad del array actual, cuando esta se agote.

```
class VectorInt {
...
private:
    /**
     * Pointer to a dynamic array of integers
     */
    int* _values;

    /**
     * Number of elements contained in the dynamic array _values
     */
    int _size;

    /**
     * Maximun number of elements that can be stored in the dynamic array
     * _values
     */
    int _capacity;

    /**
     * Size of new memory blocks when resizing the dynamic array _values
     */
    static const int BLOCK_SIZE=20;
}; // end class VectorInt
```

En esta clase será necesario implementar o revisar los siguientes métodos o constructores:

- Constructor `VectorInt(int size=0)`. Se debe refactorizar debido a la necesidad de reservar memoria para el array dinámico.
- Constructor de copia. Constructor que debe implementarse en aquellas clases que reservan memoria dinámica.
- Destructor. También debe implementarse en aquellas clases que reservan memoria dinámica.
- Operador de asignación. Tercer método a implementar en clases que reservan memoria dinámica.
- Método `int getCapacity()`. Ahora, la capacidad del array está almacenada en el dato miembro `_capacity`.

- Método `bool append()`. Este método debe modificarse para que realoje el array dinámico cuando se va a insertar una nueva localización y su capacidad se ha agotado. Ahora el método no lanzará una excepción cuando la capacidad del array se ha agotado, pues esto lo resuelve el método con el realojo.

Revise el resto de métodos, por si hubiese que modificar alguno más.

Además, debería implementar métodos privados que realicen las tareas de reservar la memoria dinámica, liberarla, copia y realojo, pues pueden usarse en varios de los métodos públicos de la clase. Implementando estos métodos privados, evitamos repetición de código.

- Clase `DataSet`. Nueva clase que contiene una matriz dinámica de enteros en la que se guarda el conjunto de instancias de un dataset. El constructor de esta clase crea una matriz de enteros con el número de filas y columnas deseado. Una vez creada, no hay ningún método que permita cambiar su tamaño. Describamos brevemente los datos miembro de esta clase:

  - `static const std::string MAGIC_STRING_T`: un string que se usa como primera línea de los ficheros `dts`. Su contenido es inicializado en el fichero `DataSet.cpp`.
  - `int _nInstances`: número de filas de la matriz de enteros. Coincide con el número de instancias del dataset.
  - `int _nLocations`: número de columnas de la matriz de enteros. Coincide con el número de localizaciones del dataset.
  - `int** _values`: puntero a la matriz de enteros.
  - `VectorInt _labels`: objeto `VectorInt` que contiene las etiquetas de cada una de las intancias del dataset. El número de etiquetas debería ser igual al número de filas de la matriz.
  - `VectorLocation _locations`: objeto `VectorLocation` que contiene las localizaciones del dataset. El número de localizaciones debería ser igual al número de columnas de la matriz.

```
class DataSet {
...
private:
    /**
     * A const string with the magic string for text files
     */
    static const std::string MAGIC_STRING_T;

    /**
     * Bidimensional matrix with the values of the instances of this DataSet.
     * It is a 2D matrix of integer values, with n rows and m columns. n is
     * the number of instances in the DataSet and m is the number of locations.
     */
    int** _values;

    /**
     * Number of rows in the 2D matrix. It is the number of instances of the
     * DataSet
     */
    int _nInstances;

    /**
     * Number of columns of the 2D matrix. It is the number of
     * locations of the DataSet
     */
    int _nLocations;
```

```
    /**
     * A VectorInt (vector of integers) with the labels of each one of the
     * n instances in this DataSet. The order of the integers in this vector
     * is assumed to correspond to the order of the rows in the 2D matrix of
     * this DataSet.
     */
    VectorInt _labels;

    /**
     * A VectorLocation object containing the m locations used in this DataSet.
     * The order of the locations in this vector is assumed to correspond to
     * the order of the columns in the 2D matrix of this DataSet.
     */
    VectorLocation _locations;
}; // end class DataSet
```

Implemente en esta clase los métodos que aparecen en `DataSet.h` (ver sección 6.5). Además, debería implementar métodos privados que realicen las tareas de reservar la memoria dinámica, liberarla y copia, pues pueden usarse en varios de los métodos públicos de la clase. Implementando estos métodos privados, evitamos repetición de código. En este caso, no necesitamos un método para realojar, pues es algo que no necesitamos hacer en esta clase.

- Módulo `main.cpp`. Este programa tiene por finalidad lo descrito en la sección 5.1. Teniendo en cuenta los detalles comentados en tal sección y los pasos esbozados en el código proporcionado para esta función (sección 6.6), complete el código de este módulo.

Recuerde además corregir los fallos detectados en la práctica anterior, pues se procede con una nueva evaluación de cada uno de los métodos implementados, por lo que los errores no corregidos volverían a penalizar en esta práctica.

5.4. Entrega de la práctica

La práctica deberá ser entregada en Prado, durante el periodo habilitado en la propia actividad de entrega, y consistirá en un fichero ZIP del proyecto. El nombre del fichero puede ser cualquiera, pero tendrá la extensión zip. Por ejemplo, podría llamarse `PRACTICAMP.zip`. Para obtener este fichero se sugiere utilizar el script `runZipProject.sh` (ver el guion de `Fraud0`). Compruebe que lo entregado es compilable y operativo y, no olvide poner el nombre de los componentes del equipo en la cabecera del fichero `main.cpp`.

6. Código para la práctica

6.1. Location.h

```
/*
 * Metodología de la Programación
 * Curso 2025/2026
 */

/**
 * @file Location.h
 * @author Silvia Acid Carrillo <acid@decsai.ugr.es>
 * @author Andrés Cano Utrera <acu@decsai.ugr.es>
 * @author Luis Castillo Vidal <L.Castillo@decsai.ugr.es>
 *
 * Created on 6 de octubre de 2025, 12:27
 */

#ifndef LOCATION_H
#define LOCATION_H

#include <iostream>
#include <string>

/**
 * @class Location
 * @brief It stores data about a location where people spend money on some
 * service or product. A location contains data about the x and y position
 * (double values) in the 2D space and the name of the location
 * (a string that can contain several words).
 */
class Location {
public:
    /**
     * @brief Constructor that builds a Location object with a value of 0
     * for the x and y coordinates, and an empty string for the name.
     */
    Location();

    /**
     * @brief Returns the x-coordinate of this location
     * Query method
     * @return The x-coordinate of this location
     */
    double getX() const;

    /**
     * @brief Returns the y-coordinate of this location
     * Query method
     * @return The y-coordinate of this location
     */
    double getY() const;

    /**
     * @brief Returns the name of this location
     * Query method
     * @return The name of this location
     */
    std::string getName() const;

    /**
     * @brief Checks if the position of this object is inside the area
     * determined by the two given locations: Location A (@p bottomLeft) and
     * B (@p topRight).
     *
     *      +------------+(B)
     *      |            |
     *      |            |  +(O3)
     *      |            |
     *      |     +(O2)  |
     *      |            |
     *      |            |
     *   (A)+------------+
     *          +(O1)
     *
     * Examples of returning false:
     * - Object O3 is outside the given area (O3.x > B.x)
     * - Object O1 is outside the given area (O1.y < A.y)
     * Example of returning true:
     * - Object O2 is inside the given area
     * - Object A is inside the given area
     * - Object B is inside the given area
     *
     * Query method.
     * @param bottomLeft The Location of the bottom left point. Input parameter
     * @param topRight The Location of the top right point. Input parameter
     * @return true if the Location of this object is inside the area
     * determined by the two given Locations
     */
    bool isInsideArea(const Location &bottomLeft, const Location &topRight) const;

    /**
     * @brief Calculates the square of the Euclidean distante from this location
     * to the provided location. That is, it loc1 and loc2 are two
     * Location objects, then this method returns:
     * \f$ (loc1.x - loc2.x)^2 + (loc1.y - loc2.y)^2 \f$
```

```
     * Query method
     * @param location A Location object. Input parameter
     * @return Returns the square of the Euclidean distante from this
     * location to the provided location.
     */
    double squaredDistance(const Location& location) const;

    /**
     * @brief Calculates the Euclidean distante from this location to the provided
     * location. That is, it loc1 and loc2 are two Location objects, then
     * this method returns:
     * \f$ \sqrt{ (loc1.x - loc2.x)^2 + (loc1.y - loc2.y)^2 } \f$
     * Note that this method can be implemented using the squaredDistance()
     * method
     * Query method
     * @param location A Location object. Input parameter
     * @return The Euclidean distante from this location to the provided
     * location.
     */
    double distance(const Location& location) const;

    /**
     * @brief Obtains a string with the x and y coordinates
     * and the name (whitespace separated). If the name is an empty string,
     * then only the x and y coordinates are included in the resulting string.
     * To convert the x and y coordinates to a string you must use the
     * std::to_string(int) C++ function.
     * Query method
     * @return string with information about this Location object
     */
    std::string toString() const;

    /**
     * @brief Sets the x-coordinate in this object
     * Modifier method
     * @param x The new value for the x-coordinate. Input parameter
     */
    void setX(double x);

    /**
     * @brief Sets the y-coordinate in this object
     * Modifier method
     * @param y The new value for the y-coordinate. Input parameter
     */
    void setY(double y);

    /**
     * @brief Sets the name in this object
     * Modifier method
     * @param name A string with the new value for the name. Input parameter
     */
    void setName(const std::string& name);

    /**
     * @brief Sets the x and y coordinates, and the the name in this object
     * Modifier method
     * @param x The new value for the x-coordinate. Input parameter
     * @param y The new value for the y-coordinate. Input parameter
     * @param name A string with the new value for the name. Input parameter
     */
    void set(double x, double y, const std::string& name);

    /**
     * @brief Reads from the provided input stream the information
     * to fill this Location object.
     * @note This method reads two double values (space separated) from the
     * input stream that are used to set the x and y coordinates of this object.
     * Then, it reads the string that can be formed from the current position of
     * the input stream to the end of the line. This string is then trimmed
     * (see the Trim(string) function) to remove its spaces at the beginning
     * and at the end. The resulting string is used to set the name of this
     * Location object. Note that the name can have several words separated by
     * whitespaces.
     * @param is Input stream. Input/output parameter
     */
    void load(std::istream& is);

private:
    /**
     * The x-coordinate of this location
     */
    double _x;

    /**
     * The y-coordinate of this location
     */
    double _y;

    /**
     * The name of this location. It can contain several words.
     */
    std::string _name;
}; // end of class Location

/**
 * Removes spaces and \t characters at the beginning and at the end of the
 * provided string @p myString. If the provided string @p myString is empty
 * or contains only spaces or \t characters then @p myString will contain an
 * empty string after calling to this function.
 * @note This function can be easily implemented using the methods
```

```
 * find_first_not_of(string) and find_last_not_of(string) of class string.
 * @param myString a string. Input/Output parameter
 */
void Trim(std::string & myString);

#endif /* LOCATION_H */
```

6.2. VectorLocation.h

```
/*
 * Metodología de la Programación
 * Curso 2025/2026
 */

/**
 * @file VectorLocation.h
 * @author Silvia Acid Carrillo <acid@decsai.ugr.es>
 * @author Andrés Cano Utrera <acu@decsai.ugr.es>
 * @author Luis Castillo Vidal <L.Castillo@decsai.ugr.es>
 *
 * Created on 30 de julio de 2025, 11:27
 */

#ifndef VECTORLOCATION_H
#define VECTORLOCATION_H

#include "Location.h"

/**
 * @class VectorLocation
 * @brief A VectorLocation object contains a vector of Location objects. It has
 * a capacity (maximun number of Locations that can be stored in the vector)
 * and a size (number of Locations the vector currently contains).
 * The public methods of this class do not allow a VectorLocation to contain two
 * Location objects with identical names.
 */
class VectorLocation {
public:
    /**
     * @brief It builds a VectorLocation object (vector of Location objects)
     * with a size and capacity equal to the provided value (@p size).
     * Each element in the vector is initialized with the default Location
     * constructor.
     * @throw std::out_of_range Throws a std::out_of_range exception if
     * @p size < 0
     * @param size The size for the vector of Location in this object. Input
     * parameter
     */
    VectorLocation(int size=0);

    /**
     * @brief Copy constructor
     * @param orig the VectorLocation object used as source for the copy.
     * Input parameter
     */
    VectorLocation(VectorLocation orig);

    /**
     * @brief Destructor
     */
    ~VectorLocation();

    /**
     * @brief Overloading of the assignment operator for VectorLocation class
     * Modifier method
     * @param orig the VectorLocation object used as source for the assignment.
     * Input parameter
     * @return A reference to this object
     */
    VectorLocation operator=(VectorLocation orig);

    /**
     * @brief Gets the number of elements in the vector of this object
     * Query method
     * @return The number of elements
     */
    int getSize() const;

    /**
     * @brief Gets the capacity of the vector in this object
     * Query method
     * @return The capacity of the vector in this object
     */
    int getCapacity() const;

    /**
     * @brief Obtains a string with information about this VectorLocation object,
     * in the following format:
     * - First line, the number of Location objects in this vector, converted to
     * a string (using the to_string(int) C++ function).
     * - For each Location, a line with its information, converted to
     * a string with the Location::toString() method.
     *
     * For example, the following is the content of the string for a
     * VectorLocation object with 4 locations:
4
24.8 14.9 Quadrangle
25.6 14.9 Ivy
26.4 14.9 Cottage
27.3 14.5 Cap & Gown
     * Query method
     * @return string with information about this VectorLocation object
     */
    std::string toString() const;

    /**
```

```
 * @brief Searches the provided Location in the array of locations in this
 * object. If found, it returns the position where it was found. If not,
 * it returns -1. We consider that position 0 is the first location in the
 * list of locations and this->getSize()-1 the last location.
 * In order to find a location consider only equality in the name field.
 * Query method
 * @param location A Location. Input parameter
 * @return If found, it returns the position where the location
 * was found. Otherwise it returns -1
 */
int findLocation(const Location& location) const;

/**
 * @brief Returns a VectorLocation object with those locations whose
 * positions are inside the area determined by the two given Locations.
 * Query method
 * @param bottomLeft The Location of the bottom left point. Input parameter
 * @param topRight The Location of the top right point. Input parameter
 * @return A VectorLocation with the selected Locations.
 */
VectorLocation select(const Location & bottomLeft,
    const Location &topRight) const;

/**
 * @brief Removes all the elements in this object, leaving the container
 * with a size equal to 0. It only needs to set the number of elements
 * (_size field) to zero.
 * Modifier method
 */
void clear();

/**
 * @brief Gets a const reference to the Location element at the given
 * position
 * Query method
 * @throw std::out_of_range Throws an std::out_of_range exception if the
 * given position is not valid.
 * @param pos position in the VectorLocation object. Input parameter
 * @return A const reference to the Location element at the given position
 */
const Location & at(int pos) const;

/**
 * @brief Gets a reference to the Location element at the given position.
 * Modifier method
 * @throw std::out_of_range Throws an std::out_of_range exception if the
 * given position is not valid
 * @param pos position in the VectorLocation object. Input parameter
 * @return A reference to the Location element at the given position.
 */
Location & at(int pos);

/**
 * @brief Appends a copy of the given Location object at the first free
 * position in the array of Location in this object. The location is only
 * appended to this object if it was not already found in this object or
 * its name is an empty string.
 * If the dynamic array of Location was full (its capacity was full), this
 * method automatically reallocates a new array with a capacity equal to
 * the current capacity plus an extra block of size equal to BLOCK_SIZE.
 * Modifier method
 * @param value the new Location object to be appended. Input parameter
 * @return true if the given Location could be inserted in this
 * VectorLocation object; false otherwise (the location was already found
 * in this object)
 */
bool append(const Location& location);

/**
 * @brief Appends to this VectorLocation object, the list of
 * Location objects contained in the provided VectorLocation object
 * that are not found (using VectorLocation::findLocation(Location)) in
 * this object.
 * This method could be implemented with the help of the method
 * VectorLocation::append(const Location & location), to append to this
 * object, the Locations of the provided VectorLocation object.
 * Modifier method
 * @param crimeSet A VectorLocation object. Input parameter
 */
void join(const VectorLocation& locations);

/**
 * Sorts the array of locations in this object by increasing alphabetical
 * order of the name of its location (a string).
 * Modifier method
 */
void sort();

/**
 * @brief Gets the position in this vector of the Location object nearest to
 * the provided location
 * Query method
 * @param location A Location object. Input parameter
 * @return the position of the Location object nearest to the provided
 * location.
 * If returns -1 if this vector is empty
 */
int nearest(Location location);

/**
```

```
     * Assigns the provided value to all the elements in this vector
     * Modifier method
     * @param location A Location object. Input parameter
     */
    void assign(Location location);

    /**
     * @brief Reads from the provided input stream the information
     * to fill this VectorLocation object. See files *.loc in the folder
     * DataSets as examples of this kind of file.
     * @note This method should remove any Location previously contained in the
     * provided VectorLocation object.
     * @note This operator throws an exception in some error cases (see below).
     * Before throwing the corresponding exception, this method clears
     * the object (it calls to clear() method) to leave the object in a
     * consistent state.
     * Modifier method
     * @throw std::out_of_range Throws a std::out_of_range exception if the
     * number of Location read from the input stream is negative.
     * @param is Input stream. Input/output parameter
     */
    void load(std::istream is);

private:
    /**
     * Pointer to a dynamic array of Locations
     */
    Location* _locations;

    /**
     * Number of Location objects contained in the dynamic array _locations
     */
    int _size;

    /**
     * Maximun number of Location objects that can be stored in the dynamic array
     * _locations
     */
    int _capacity;

    /**
     * Size of new memory blocks when resizing the dynamic array _locations
     */
    static const int BLOCK_SIZE=20;
}; // end of class VectorLocation

#endif /* VECTORLOCATION_H */
```

6.3. VectorInt.h

```cpp
/*
 * Metodología de la Programación
 * Curso 2025/2026
 */

/**
 * @file VectorInt.h
 * @author Silvia Acid Carrillo <acid@decsai.ugr.es>
 * @author Andrés Cano Utrera <acu@decsai.ugr.es>
 * @author Luis Castillo Vidal <L.Castillo@decsai.ugr.es>
 *
 * Created on 30 de julio de 2025, 9:54
 */

#ifndef VECTORINT_H
#define VECTORINT_H

#include <iostream>
#include <string>

/**
 * @class VectorInt
 * @brief A VectorInt object contains a vector of integer elements. It has
 * a capacity (maximun number of elements that can be stored in the vector)
 * and a size (number of elements the vector currently contains).
 */
class VectorInt {
public:

    /**
     * @brief It builds a VectorInt object (vector of integers) with a
     * size and capacity equal to the provided value (@p size). Each element
     * will be filled with a value equal to 0.
     * @throw std::out_of_range Throws a std::out_of_range exception if
     * @p size < 0
     * @param size The size for the vector of integers in this object. Input
     * parameter
     */
    VectorInt(int size=0);

    /**
     * @brief Copy constructor
     * @param orig the VectorInt object used as source for the copy. Input
     * parameter
     */
    VectorInt(VectorInt orig);

    /**
     * @brief Destructor
     */
    ~VectorInt();

    /**
     * @brief Overloading of the assignment operator for VectorInt class
     * Modifier method
     * @param orig the VectorInt object used as source for the assignment. Input
     * parameter
     * @return A reference to this object
     */
    VectorInt operator=(VectorInt orig);

    /**
     * @brief Gets the number of elements in the vector of this object
     * Query method
     * @return The number of elements
     */
    int getSize();

    /**
     * @brief Gets the capacity of the vector in this object
     * Query method
     * @return The capacity of the vector in this object
     */
    int getCapacity();

    /**
     * @brief Compares the integer vectors of this object and the provided
     * object, and returns the number of identical elements in both of them. The
     * comparison is performed in order, element by element, in both vectors.
     * For example, given the following two vectors:
     * 2 1 2 3 5
     * 1 1 2 4 5
     * this method will return 3 (there is a match in positions 1, 2 and 4)
     * @throw std::invalid_argument Throws an std::invalid_argument exception
     * if the sizes of this and the provided object are different
     * Query method
     * @param other A VectorInt object. Input parameter
     * @return The number of identical elements in the vectors of this
     * and the provided object.
     */
    int countIdenticalElements(VectorInt other);


    /**
     * @brief Obtains a string with information about this VectorInt object,
     * in the following format:
     * - First line, the number of elements in this vector.
```

```
     * - Second line, the elements in this vector, separated by a whitespace.
     * Take into account that a '\n' should appear after the last element
     * instead of a whitespace.
     * Query method
     * @return string with information about this VectorInt object
     */
    std::string toString();

    /**
     * @brief Gets the Euclidean distance between this and the provided object.
     * The Euclidean distance between two points \f$ P=(p_1, p_2, ... , p_n) \f$
     * and \f$ Q=(q_1, q_2, ... , q_n) \f$ in an n-dimensional space \f$ R^n \f$
     * is the length of the line segment connecting them, calculated as the
     * square root of the sum of the squared differences of their components:
     * \f$ d(P,Q)=\sqrt{ \sum_{i=1}^{n} (p_i-q_i)^{2} } \f$
     * @throw std::invalid_argument Throws an std::invalid_argument exception
     * if the size of this object and the provided one are not equal
     * @throw std::invalid_argument Throws an std::invalid_argument exception
     * if the size of the provided object is zero
     * Query method
     * @param other A VectorInt. Input parameter
     * @return The Euclidean distance between this and the provided objects
     */
    double distance(VectorInt other);

    /**
     * @brief Assigns the provided value to all the elements in this vector
     * Modifier method
     * @param value An integer value. Input parameter
     */
    void assign(int value=0);

    /**
     * @brief Appends the given integer value at the end (first free position)
     * of the array of integers in this object.
     * If the dynamic array of integers was full (its capacity was full), this
     * method automatically reallocates a new array with a capacity equal to
     * the current capacity plus an extra block of size equal to BLOCK_SIZE.
     * Modifier method
     * @param value the new integer value to be appended. Input parameter
     */
    void append(int value);

    /**
     * @brief Removes all the elements in this object, leaving the container
     * with a size equal to 0. It only need to set the number of elements
     * (_size field) to zero.
     * Modifier method
     */
    void clear();

    /**
     * @brief Gets a const reference to the integer element at the given
     * position
     * Query method
     * @param pos position in the VectorInt object. Input parameter
     * @throw std::out_of_range Throws an std::out_of_range exception if the
     * given position is not valid.
     * @return A const reference to the integer element at the given position
     */
    int at(int pos);

    /**
     * @brief Gets a reference to the integer element at the given position.
     * Modifier method
     * @param pos position in the VectorInt object. Input parameter
     * @throw std::out_of_range Throws an std::out_of_range exception if the
     * given position is not valid
     * @return A reference to the integer element at the given position.
     */
    int at(int pos);

private:
    /**
     * Pointer to a dynamic array of integers
     */
    int* _values;

    /**
     * Number of elements contained in the dynamic array _values
     */
    int _size;

    /**
     * Maximun number of elements that can be stored in the dynamic array
     * _values
     */
    int _capacity;

    /**
     * Size of new memory blocks when resizing the dynamic array _values
     */
    static const int BLOCK_SIZE=20;

}; // end of class VectorInt

#endif /* VECTORINT_H */
```

6.4. Clustering.h

```
/*
 * Metodología de la Programación
 * Curso 2025/2026
 */

/**
 * @file Clustering.h
 * @author Silvia Acid Carrillo <acid@decsai.ugr.es>
 * @author Andrés Cano Utrera <acu@decsai.ugr.es>
 * @author Luis Castillo Vidal <L.Castillo@decsai.ugr.es>
 *
 * Created on 22 de octubre de 2025, 11:57
 */

#ifndef CLUSTERING_H
#define CLUSTERING_H

#include "VectorInt.h"
#include "VectorLocation.h"

/**
 * @class Clustering
 * @brief This class is used to obtain a clustering of a given set of locations
 * (Location objects). A clustering algorithm groups the input locations
 * into K disjoint groups (clusters). The clustering algorithm used in this
 * class is the K-Means algorithm. Each group will consist of a disjunct subset
 * of the original location set and a representative location (centroid), which
 * is calculated as the vector of the means of the locations in the group. The
 * run() method in this class executes the algorithm. An example of use is the
 * following:
VectorLocation locations;
// Suppose here code to fill the vector of locations
Clustering clustering;
clustering.set(locations, 5);
clustering.run();
 */
class Clustering {
public:
    /**
     * @brief Constructor that builds a Clustering object initializing
     * the field (_K) for the number of clusters with 0; the field (_isDone)
     * that tells whether the run() method has already been executed
     * with false; the fields _sumWCV and _numIterations with 0 and the field
     * _seed with DEFAULT_RANDOM_SEED.
     * The fields (_locations, _clusters, _centroids) are initialized with
     * the default constructor of their classes.
     */
    Clustering();

    /**
     * @brief Gets the number of clusters
     * Query method
     * @return The number of clusters
     */
    int getK();

    /**
     * @brief Gets the current vector of centroids
     * Query method
     * @return A const reference to the vector of centroids (a VectorLocation
     * object).
     */
    VectorLocation getCentroids();

    /**
     * @brief Indicates whether the clustering algorithm (run() method) has
     * already been executed for this Clustering object.
     * Query method
     * @return true if the run() method has already been executed for this
     * Clustering object; false otherwise
     */
    bool isDone();

    /**
     * @brief Gets the number of locations in the set of locations of this
     * Clustering object.
     * Query method
     * @return The number of locations in the set of locations of this
     * Clustering
     */
    int getNumLocations();

    /**
     * @brief Gets the cluster number for the Location determined by the
     * provided integer (@p locationIndex)
     * Query method
     * @param locationIndex An integer that determines the position of a
     * location in the vector of Location objects. Input parameter
     * @return The cluster number for the Location determined by
     * the provided integer (@p locationIndex). If the
     * clustering algorithm has not been run (_isDone is false), then it returns
     * -1.
     */
    int clusterOf(int locationIndex);

    /**
     * @brief Gets the value of the sum of the within-cluster variances of this
```

```
     * Clustering. This value is obtained from the field _sumWCV
     * @note The returned value will likely be a garbage value if the run()
     * method has not yet been executed (_isDone is false)
     * Query method
     * @return A double with the sum of within-cluster variances of this
     * Clustering
     */
    double getSumWCV();

    /**
     * @brief  Gets the number of iterations used in the KMeans
     * algorithm until no cluster assignment change occurs.
     * It obtains the number of iterations (field _numberIterations) used
     * in the KMeans algorithm until no cluster change occurred.
     * @note The returned value will likely be a garbage value if the run()
     * method has not yet been executed (_isDone is false)
     * Query method
     * @return The number of iterations used in the KMeans algorithm
     */
    int getNumIterations();

    /**
     * @brief  Gets a string with information about the provided cluster (index
     * of a cluster). That string contains a line for each location in the given
     * cluster. The information of each location consists of an integer with
     * its location index (whithin the _location field), a whitespace, and
     * the information returned by the Location::toString() method for that
     * location. An example is the following (a cluster with three locations):
0 3.300000 9.100000 Graduate College
6 12.600000 17.500000 Admission Information Center
7 12.500000 20.000000 Rockefeller College
     * Query method
     * @param cluster An integer, that defines the index of a cluster. Input
     * parameter
     * @return Gets a string with information about the provided cluster. If the
     * clustering algorithm has not been run (_isDone is false) or an
     * invalid value of cluster is provide, then it returns an empty string.
     */
    std::string clusterInfo(int cluster);

    /**
     * @brief Obtains a string with the some statistics data about this
     * Clustering object.
     * The first line contains the value of K; the second line contains the
     * value of the sum of within-cluster variances; the third line contains
     * the number of iterations used in the KMeans algorithm.
     * Query method
     * @return A string with the content described above
     */
    std::string getStatistics();

    /**
     * @brief Indicates whether this Clustering object is equivalent to the
     * provided Clustering object (@p other). Two Clustering objects are
     * considered equivalent if they have the same K value,
     * the same sum of within-cluster variances and the same number of
     * iterations in the KMeans algorithm.
     * If either this Clustering object or the provided Clustering object
     * (@p other) has not yet run the clustering algorithm (the run() method),
     * then they are not considered equivalent.
     * Query method
     * @param other A Clustering object. Input parameter
     * @return true if this Clustering object is equivalent to the provided
     * Clustering object (@p other); false otherwise
     */
    bool isEquivalentTo(Clustering other);

    /**
     * @brief Obtains a string with the fields of this Clustering object.
     * The first line contains the value of K; the second line contains the
     * value of the sum of within-cluster variances; the third line contains
     * the number of iterations used in the KMeans algorithm.
     * In the next three lines, information about the cluster number for each
     * location.
     * If follows information about each centroid
     * Next, for each cluster, information about its locations
     * Here, an example is shown:
K=5
Sum of within-cluster variances: 161.136889
Number of iterations: 5
Cluster number for each location:
21
0 2 2 2 2 2 1 1 1 1 1 4 4 4 4 4 4 4 4 4 3
Centroids:
5
3.300000 9.100000
14.880000 21.280000
12.600000 9.460000
18.500000 6.700000
26.088889 15.366667
Cluster 0 information:
0 3.300000 9.100000 Graduate College
Cluster 1 information:
6 12.600000 17.500000 Admission Information Center
7 12.500000 20.000000 Rockefeller College
8 13.700000 22.800000 Art Museum Store
9 17.000000 23.300000 Princeton University Store
10 18.600000 22.800000 Labyrinth Books
Cluster 2 information:
1 11.800000 6.900000 Wawa Public Restrooms
```

```
2 12.600000 7.300000 Princeton Station
3 10.100000 9.700000 Forbes College
4 13.300000 12.200000 Dinky Bar & Kitchen
5 15.200000 11.200000 Whitman College
Cluster 3 information:
20 18.500000 6.700000 Yeh College
Cluster 4 information:
11 20.900000 15.400000 Frist Campus Center
12 23.700000 14.800000 Cannon Dial Elm
13 24.800000 14.900000 Quadrangle
14 25.600000 14.900000 Ivy
15 26.400000 14.900000 Cottage
16 27.300000 14.500000 Cap & Gown
17 28.200000 15.000000 Cloister
18 29.100000 15.200000 Charter
19 28.800000 18.700000 Engineering Quadrangle
     * Query method
     * @return A string with the content described above
     */
    std::string toString();

    /**
     * @brief Sets the vector of locations (_locations), the value of K (_K)
     * and the seed (_seed) with the provided values.
     * The fields _isDone, _clusters and _centroids_ are also initialized
     * appropriately: _isDone is set to false; _clusters is initialized with
     * a VectorInt object with size equal to the number of locations;
     * _centroids is initialized with a VectorLocation object with size K;
     * the fields _sumWCV and _numIterations are set to 0.
     * Modifier method
     * @throw std::invalid_argument Throws an std::invalid_argument exception
     * if K<=0
     * @param locations The vector of locations to used in the clustering
     * algorithm. Input parameter
     * @param K The parameter K for the K-Means algorithm. Input parameter
     * @param seed The seed used to initialize the random number generator
     * (with srand(seed)) in the initialClusterAssignment() method.
     * Input parameter
     */
    void set(VectorLocation locations, int K,
        unsigned int seed=DEFAULT_RANDOM_SEED);

    /**
     * @brief Run the clustering algorithm. The clustering algorithm implemented
     * in this method is the KMeans algorithm.
     * After running this method the following postcondition should be verified:
     * @post The _centroids field should contain the vector of the means of
     * locations in each group
     * @post _clusters (a VectorInt object) should contain the cluster number
     * for each location in the vector of locations (_locations)
     * @post The _isDone field should be set to true
     * @post The _sumWCV field should be set to the sum of the within-cluster
     * variances using the claculateSumWithinClusterVariances() method.
     * @post The _numberIterations field should be set to the number of
     * iterations used by the KMeans algorithm.
     *
     * A brief description of the KMeans algorithm is the following:
     *
     * -# Perform an initial assignment of a cluster to each of the Location
     * objects. This must be done by initializing the random number
     * generator using srand(seed);
     * Then, for each location i, assign cluster number rand() % _K
     * to that location. Note that this step is done by modifying the _clusters
     * field.
     * -# Repeat while any location changes its cluster number:
     *     - For each cluster, recalculate its centroid (update _centroids
     *       field).
     *     - Assign each location to a cluster (update _clusters field).
     *       Each location is assigned to the cluster that provides
     *       the shortest distance to its centroid.
     *
     * Modifier method
     */
    void run();

private:
    /**
     * Default random seed used to initialize the random number
     * generator used in the initialClusterAssignment() method.
     */
    static const unsigned int DEFAULT_RANDOM_SEED=1761560597U;

    /**
     * Vector of Location objects. This will be the input for the
     * clustering algorithm
     */
    VectorLocation _locations;

    /**
     * Number of clusters
     */
    int _K;

    /**
     * Vector of n integers (number of elements in _locations). This will
     * be the output of the clustering algorithm.
     * The integers in this vector define the cluster number (values in the
     * interval [0 - _K-1]) for each one of the Location object in _locations
     */
    VectorInt _clusters;
```

```
    /**
     * A vector of @p _K Location objects with the centroids of each cluster.
     */
    VectorLocation _centroids;

    /**
     * It indicates whether the run() method has already been executed to
     * obtain a clustering of the set of Location objects. The run() method
     * should set this field to true when it ends its execution.
     */
    bool _isDone;

    /**
     * Seed used to initialize the random number generator (with srand(seed)) in
     * the initialClusterAssignment() method.
     */
    unsigned int _seed;

    /**
     * This is the sum of the whithin-cluster variances of the clusters in
     * this Clustering object.
     * The value for this field should be set at the end of run() method
     * with the method calculateSumWCV(). Before running the
     * run() method, this field will likely contain a garbage value.
     * @note The value of the sum of the within-cluster variances can also be
     * retrieved using the method calculateSumWCV().
     * This field is introduced in this class to calculate its value once,
     * and thus avoiding having to recalculate it every time it
     * is needed
     */
    double _sumWCV;

    /**
     * This field represents the number of iterations used in the KMeans
     * algorithm until no cluster change occurred. Before running the
     * run() method, this field will likely contain a garbage value.
     * The value for this field should be set at the end of run() method.
     */
    int _numIterations;

    /**
     * @brief Performs an initial assignment of a cluster to each one of the
     * Location objects. This method begins by initializing the random number
     * generator using:
     * srand(seed);
     * Then, for each location i, assign cluster number rand() % _K to that
     * location.
     * Modifier method
     */
    void initialClusterAssignment();

    /**
     * @brief Calculates the sum of the within-cluster variances of this
     * Clustering object.
     * The within-cluster variance of a cluster C_j is calcultated with:
     * $\sum_{p_i \in C_j} squaredDistance(p_i, centroid_{C_j})$
     * @return A double with the sum of within-cluster variances of this
     * Clustering
     * Modifier method
     */
    double calculateSumWCV();
}; // end of class Clustering

#endif /* CLUSTERING_H */
```

6.5. DataSet.h

```
/*
 * Metodología de la Programación
 * Curso 2025/2026
 */

/**
 * @file DataSet.h
 * @author Silvia Acid Carrillo <acid@decsai.ugr.es>
 * @author Andrés Cano Utrera <acu@decsai.ugr.es>
 * @author Luis Castillo Vidal <L.Castillo@decsai.ugr.es>
 *
 * Created on 30 de julio de 2025, 10:00
 */

#ifndef DATASET_H
#define DATASET_H

#include <string>
#include <iostream>
#include <fstream>

#include "VectorInt.h"
#include "VectorLocation.h"
#include "Clustering.h"

/**
 * @class DataSet
 * @brief A DataSet is an object that contains a set of n instances. Each
 * instance contains a vector of m integers that represents the amount of
 * money spent by a person in each one of m locations.
 * This class uses a bidimensional matrix with n rows and m columns to store
 * the values of the set of instances.
 * This class also contains a vector of n labels (VectorInt _labels) to store the
 * label of each instance. Each label is an integer that defines the class
 * value of the corresponding instance.
 * Finally, this class contains a vector of m Location objects
 * (VectorLocation _locations) to store each locations in the DataSet.
 * In the 2D matrix, each column i is associated with location i in the
 * _locations vector.
 */
class DataSet {
public:
    /**
     * @brief It builds a DataSet object with the provided number of instances
     * and number of locations. This class uses a bidimensional matrix with
     * nInstances rows and nLocations columns to store the values of the set
     * of instances.
     * Note that this constructor also builds the vector (with a size equal to
     * nInstances) of labels and the vector of locations (with a size equal to
     * nLocations).
     * This constructor initializes with a value of 0 all the values in the 2D
     * matrix.
     * The vector of labels is initialized with a value of 0 for each instance.
     * The Location objects in the vector of locations are built with the
     * Location default constructor.
     * @param nInstances An integer with the number of instances.
     * Input parameter
     * @param nLocations An integer with the number of locations.
     * Input parameter
     */
    DataSet(int nInstances = 0, int nLocations = 0);

    /**
     * @brief Copy constructor
     * @param orig the DataSet object used as source for the copy.
     * Input parameter
     */
    DataSet(DataSet orig);

    /**
     * @brief Destructor
     */
    ~DataSet();

    /**
     * @brief Overloading of the assignment operator for DataSet class
     * Modifier method
     * @param orig the DataSet object used as source for the assignment.
     * Input parameter
     * @return A reference to this object
     */
    DataSet operator=(DataSet orig);

    /**
     * @brief Gets the number of instances in this DataSet
     * Query method
     * @return The number of instances in this DataSet
     */
    int getNumInstances();

    /**
     * @brief Gets the number of localizations in this DataSet
     * Query method
     * @return The number of localizations in this DataSet
     */
    int getNumLocations();
```

```
/**
 * @brief Gets the value for the instance instanceIndex at the localization
 * locationIndex.
 * @throw Throws a std::out_of_range exception if instanceIndex is not
 * a valid index for an instance in this DataSet.
 * @throw Throws a std::out_of_range exception if locationIndex is not a
 * valid index for an location in this DataSet.
 * Query method
 * @param instanceIndex An integer with the index of the selected instance.
 * Input parameter
 * @param locationIndex An integer with the index of the selected location.
 * Input parameter
 * @return The value for the instance instanceIndex at the localization
 * locationIndex
 */
int getValue(int instanceIndex, int locationIndex);

/**
 * @brief Gets the label (integer value) of the instance at the provided
 * position (instanceIndex)
 * @throw Throws a std::out_of_range exception if instanceIndex is not
 * a valid index for an instance in this DataSet.
 * Query method
 * @param instanceIndex An integer with the index of the selected instance.
 * Input parameter
 * @return The label (integer value) of the instance at the provided
 * position
 */
int getLabel(int instanceIndex);

/**
 * @brief Gets a const reference to the vector of Location objects in this
 * DataSet
 * Query method
 * @return A const reference to the vector of Location objects in this
 * DataSet
 */
VectorLocation getVectorLocation();

/**
 * @brief Gets a const reference to the vector of labels in this DataSet
 * Query method
 * @return A const reference to the vector of labels in this DataSet
 */
VectorInt getVectorLabels();

/**
 * @brief Obtains a string with information about this DataSet object,
 * in the following format:
 * -Information about the set of locations of this DataSet:
 *    - A line with the number of locations
 *    - For each location, a line with its information, converted to
 *      a string with the Location::toString() method.
 * - Information about the labels of each instance in this DataSet:
 *    - A line with the number of instances
 *    - A line with the labels (space separated) of the set of instances.
 * - Instances of the Dataset. This is the information contained in the
 *   2D matrix of this DataSet.
 *     - For each instance, a line with the values (space separated) of
 *       that instance.
 * Query method
 * @return string with information about this CrimeSet object
 */
std::string toString() ;

/**
 * @brief Sets a new value for the instance instanceIndex at the
 * localization locationIndex.
 * Modifier method
 * @param instanceIndex An integer with the index of the selected instance.
 * Input parameter
 * @param locationIndex An integer with the index of the selected location.
 * Input parameter
 * @param value An integer with the new value for the selected instance.
 * Input parameter
 */
void setValue(int instanceIndex, int locationIndex, int value);

/**
 * @brief Sets a new the label for the provided instance.
 * Modifier method
 * @param instanceIndex An integer with the index of the selected instance.
 * Input parameter
 * @param label An integer with the new value for the label of the selected
 * instance.
 * Input parameter
 */
void setLabel(int instanceIndex, int label);

/**
 * Assigns the provided value to each instance and location. That is, all
 * instances of this DataSet are initialized with the provided value.
 * Modifier method
 * @param value An integer with the value to assign to every instance and
 * location
 * Input parameter
 */
void initInstances(int value = 0);

/**
```

```
     * @brief Removes all the information in this DataSet object:
     * - The vector of Location objects (_locations) is left empty. That is,
     * it should represent a vector with 0 locations.
     * - The vector of labels (_labels) is left empty. That is, it should
     * represent a vector with 0 labels.
     * - The matrix (_values) with the values for each instance is left empty.
     * That is, it should represent a matrix with 0 instances and 0 locations.
     * - The remaining fields of this object are updated accordingly.
     * Modifier method
     */
    void clear();

    /**
     * @brief Saves this DataSet object in the given file. See files *.dts in
     * the folder DataSets as example of files with DataSet information.
     * Note that the format of the output file is the same as the string that
     * returns the toString() method, except that the save() method should
     * insert the magic string at the beginning of the output file.
     * @throw std::ios_base::failure Throws a std::ios_base::failure exception
     * if the given file cannot be opened or if an error occurs while writing
     * to the file.
     * Query method
     * @param fileName The name of the file where the DataSet will be saved.
     * Input parameter
     */
    void save(std::string fileName);

    /**
     * @brief Loads into this object the DataSet information stored in the
     * given file. See files *.dts in the folder DataSets as example of files
     * with DataSet information.
     * @note Note that this method should remove any previous information that
     * this object previously contained.
     * @note This method throws an exception in some error cases (see below).
     * Before throwing the corresponding exception, this method clears
     * the object (it calls to clear() method) to leave the object in a
     * consistent state.
     * @throw throw std::invalid_argument Throws a std::invalid_argument
     * exception if an invalid magic string is found in the given file
     * @throw std::out_of_range Throws a std::out_of_range exception if the
     * number of locations or the number of instances in the given file is
     * negative.
     * Modifier method
     * @param fileName The name of the file where the DataSet is stored.
     * Input parameter
     * @throw std::ios_base::failure Throws a std::ios_base::failure exception
     * if the given file cannot be opened or if an error occurs while reading
     * from the file.
     */
    void load(std::string fileName);

    /**
     * @brief Gets a new DataSet from this DataSet. The DataSet will contain
     * the same number of instances as this DataSet, but the number of locations
     * is given by the number of clusters in the provided Clustering object.
     * Therefore, the returned DataSet will have a reduced number of columns
     * and an equal number of rows compared to this DataSet.
     * The provided Clustering object defines how to reduce the set of columns.
     * Column 0 will contain the sum of the columns of this DataSet belonging
     * to cluster 0, column 1 the sum of the columns belonging to cluster 1,
     * and so on.
     * The vector of locations will be a copy of the vector of centroids of the
     * provided Clustering object.
     * The vector of labels will be a copy of the vector of labels in this
     * DataSet.
     * @throw throw std::invalid_argument Throws a std::invalid_argument
     * exception if the method Clustering::isDone() return false with the
     * provided Clustering object.
     * @throw throw std::invalid_argument Throws a std::invalid_argument
     * exception if the number of locations in the provided Clustering object
     * is not equal to the number of locations in this DataSet.
     * Query method
     * @param clustering A Clustering object.
     * Input parameter
     * @return A reduced DataSet
     */
    DataSet getReducedDataSet(Clustering clustering);

private:
    /**
     * A const string with the magic string for text files
     */
    static const std::string MAGIC_STRING_T;

    /**
     * Bidimensional matrix with the values of the instances of this DataSet.
     * It is a 2D matrix of integer values, with n rows and m columns. n is
     * the number of instances in the DataSet and m is the number of locations.
     */
    int** _values;

    /**
     * Number of rows in the 2D matrix. It is the number of instances of the
     * DataSet
     */
    int _nInstances;

    /**
     * Number of columns of the 2D matrix. It is the number of
     * locations of the DataSet
```

```
     */
    int _nLocations;

    /**
     * A VectorInt (vector of integers) with the labels of each one of the
     * n instances in this DataSet. The order of the integers in this vector
     * is assumed to correspond to the order of the rows in the 2D matrix of
     * this DataSet.
     */
    VectorInt _labels;

    /**
     * A VectorLocation object containing the m locations used in this DataSet.
     * The order of the locations in this vector is assumed to correspond to
     * the order of the columns in the 2D matrix of this DataSet.
     */
    VectorLocation _locations;
}; // end of class DataSet

#endif /* DATASET_H */
```

6.6. main.cpp

```cpp
/*
 * Metodología de la Programación
 * Curso 2025/2026
 */

/**
 * @file main.cpp
 * @author Silvia Acid Carrillo <acid@decsai.ugr.es>
 * @author Andrés Cano Utrera <acu@decsai.ugr.es>
 * @author Luis Castillo Vidal <L.Castillo@decsai.ugr.es>
 *
 * Created on 24 de octubre de 2025, 9:27
 */

#include <iostream>

#include "DataSet.h"
#include "Clustering.h"

using namespace std;

/**
 * Shows help about the use of this program in the given output stream
 * @param outputStream The output stream where the help will be shown (for example,
 * cout, cerr, etc)
 * @param message Additional message to show with the help
 */
void showHelp(std::ostream& outputStream, const string &message) {
    outputStream << "ERROR in Fraud3 parameters. " << message << endl;
    outputStream << "Run with the following arguments:" << endl;
    outputStream << "Fraud3 [-K <K>] [-o <outputFile.dts>] <inputFile.dts>" <<
        endl;
    outputStream << endl;
    outputStream << "Parameters:" << endl;
    outputStream << "-K <K>: an integer with the number of clusters to use " <<
        "(5 by default)" << endl;
    outputStream << "-o <outputFile.dts>: name of the output dataset file " <<
        "(tests/output/output.dts by default)" << endl;
    outputStream << "<inputFile.dts>: name of the input dataset file" << endl;
    outputStream << endl;
}

/**
 * The purpose of this program is to read a dataset from a dts file, to reduce
 * its dimensionality using clustering, and to save the resulting dataset in
 * another dts file.
 *
 * The number of clusters K to be used in the clustering process can be provided
 * as an optional argument (if not provided, K=5 is used).
 * The name of the output dts file can be provided to the program as an optional
 * argument (if not provided, the output file is tests/output/output.dts).
 * The name of the input dts file is passed as the last argument to the program.
 * See below the Running sintax.
 *
 * The program begins by processing the command line arguments. Then, it loads
 * the input dataset from the given dts file. After that, it performs a
 * clustering of the locations in the dataset using the K-means algorithm with
 * the provided K value. Then, using the calculated clustering, it obtains a new
 * dataset with reduced dimensionality. Finally, it saves the resulting dataset
 * in the given output dts file.
 *
 * Running sintax:
 * > build/Fraud3 [-K <K>] [-o <outputFile.dts>] <inputFile.dts>
 *
 * Running example:
 * > build/Fraud3 -K 5 -o /tmp/princeton_training_reduced.dts ../Datasets/princeton_training.dts
 */
int main(int argc, char* argv[]) {
    DataSet inputDataset, // Input dataset
        outputDataset; // Output dataset with reduced dimensionality

    // Name of the output file (tests/output/output.dts by default)
    string outputFileName="tests/output/output.dts";
    int K = 5; // Number of clusters (5 by default)
    Clustering clustering; // Clustering object

    // Indicates if all the parameters starting with - has been read
    bool hasBeenReadInitialParameters = false;
    int indexInputFile = -1; // index of the input file in argv

    // Loop to process program arguments

    // Load the input dataset from the given file

    // Set the location vector and K in the clustering object. Use the default
    // seed value.

    // Run the clustering algorithm

    // Get the dataset with reduced dimensionality

    // Save the output dataset in the given file

    return 0;
}
```